Übungsblatt 4

Dr. Matthias Frank, Dr. Matthias Wübbeling, Dylan Stogianni,
Lennart Buhl, Felix Pehla, Julius Berewinkel,
Mateusz Respondek, Kilian Smeets, Jonas Reif, Till Burgheim

Ausgabe Dienstag, 03. November 2026
Abgabe bis **Freitag, 13. November 2026, 23:59 Uhr**
Vorführung vom 16. bis zum 20. November 2026

Alle Programme müssen unter **Ubuntu 24.04** kompilierbar bzw. lauffähig und ausreichend kommentiert und mit **Makefile** (C, Assembler) versehen sein, um Punkte zu erhalten. Als Compiler sollen **clang** (C) und **nasm** (Assembler) verwendet werden. Die Lösungen sind bei Ihrem Tutor während Ihrer Übungsgruppen vorzuführen. Alle Gruppenmitglieder sollten die Abgabe erklären können. Die Abgabe erfolgt mittels Ihres Git-Repositories.

Die Punkte der Aufgaben sind relevant für die Zulassung. Die Punkte der Bonusaufgaben werden auf Ihren Punktestand addiert, werden aber nicht auf die für die Zulassung benötigten Punkte addiert.

Makefile: Jede Abgabe, die Source Code beinhaltet, benötigt **ZWINGEND** ein **Makefile**! Abgaben ohne diese werden **NICHT** gewertet.

Hinweis:	Manche Browser sind nicht dazu in der Lage die in die PDFs eingebetteten Dateien anzuzeigen. Mittels eines geeigneten Readers, wie dem Adobe Reader auf Windows (Papers auf Gnome oder Okular auf KDE), lassen sich diese jedoch anzeigen und verwenden.

Aufgabe 1 Assembler Debugging mit gdb (3 Punkte)

gdb eignet sich nicht nur zum Debugging von C-Programmen, sondern bietet auch einige interessante Features zur Analyse von Assembler-Code. Hierzu eignet sich insbesondere das `text user interface` oder `tui`. Dieses ermöglicht gleichzeitigen Einblick in den Quellcode, Register oder Maschinenbefehle. Der tui-Modus lässt sich auf verschiedene Weisen aktivieren, z.B. durch Aufruf von `gdb -tui <file>` oder den Befehl `(gdb) layout <type>`. Eine Übersicht über die verfügbaren Layouts befindet sich im gdb User Manual unter TUI/Commands. Wir empfehlen `(gdb) layout reg` für das Debugging von SysProg-Übungsaufgaben. Beachten Sie, dass das ASM-Fenster standardmäßig AT&T-Syntax verwendet und mit `(gdb) set disassembly intel` auf Intel-Syntax umgeschaltet werden kann.

Korrigieren Sie nun mithilfe gdb das Programm `average.asm`, welches den (abgerundeten) Durchschnitt einiger Zahlen berechnen soll und ergänzen Sie eine Möglichkeit, die diesen Durchschnitt ausgibt:

> Hinweis: Dies wäre eine gute Anwendungsmöglichkeit der `printnumber`-Funktion aus `Hello World in Assembler` vom zweiten Übungsblatt.

```
        global  main
        section .text
main:
        lea     rsi, [array]            ; load start of array
        mov     rcx, [count]            ; load number of elements
        dec     rcx                     ; n-th element = array[n-1]
accumulate:
        mov     rax, [rsi+rcx*4]        ; load i-th element
        add     [sum], rax              ; accumulate sum as we go
        loop    accumulate              ; more numbers?
average:
        mov     rax, [sum]              ; prepare dividend
        div     qword [count]           ; rax is integer quotient, rdx is remainder
        ret

        section .data
array:  dq      89, 10, 67, 1, 4, 27, 12, 34, 85, 1
count:  dq      10
sum:    dq      0
```

Tipp: Sie können das Programm folgendermaßen kompilieren/assemblieren und linken:

```
nasm -f elf64 -g -F dwarf -o average.o average.asm
ld -e main -o average average.o
```

> Hinweis: Üblicherweise wird in NASM das Symbol `_start` für den Einstiegspunkt in ein Programm verwendet. Der Linker (ld) erlaubt aber die Spezifizierung beliebiger Labels mit dem Parameter `-e`. Überlegen Sie, für welche Anwendungsfälle dies nützlich sein könnte.

Aufgabe 2 Matrixaddition (4 Punkte + 2 Bonuspunkte)

Betrachten Sie das folgende C-Programm, welches Matrizen addiert:

```c
#include <stdlib.h>
#include <stdio.h>

void print_matrix(int n, int m, int* M);
void m_add(int n, int m, int* M1, int* M2, int* M);

int main() {
        int A[] = { /* 3x4-Matrix */
                12,2,-3,5,
                0,19,5,9,
                2,-1,13,-3
        };
        print_matrix(3, 4, A);
        int B[] = { /* 3x4-Matrix */
                30,2,-1,-47,
                0,23,2,-2,
                7,-5,-55,9
        };
        print_matrix(3, 4, B);
        int C[12]; /* 3x4-Matrix */
        m_add(3, 4, A, B, C); /* C = A + B */
        print_matrix(3, 4, C);
        return 0;
}

void print_matrix(int n, int m, int* M) {
        int x;
        int y;
        printf("\n");
        for (y=0; y < n; y++) {
                for (x = 0; x < m; x++) {
                        printf("%5d", M[y*m+x]);
                }
                printf("\n");
        }
        printf("\n");
}

void m_add(int n, int m, int* M1, int* M2, int* R) {
        int x;
        int y;
        for (y = 0; y < n; y++) {
                for (x = 0; x < m; x++) {
                        R[y*m+x] = M1[y*m+x] + M2[y*m+x];
                }
        }
}
```

Das Programm finden Sie eingebettet in die PDF-Datei.

Entfernen Sie aus diesem Programm die Funktion `void m_add(...)` und schreiben Sie eine entsprechende Funktion in einer separaten Datei in Assembler. Linken Sie anschließend die Assembler-Funktion mit dem verbleibenden C-Programm zu einer Binary und überprüfen Sie damit die korrekte Funktionsweise Ihrer Assembler-Routine. Angenommen, Ihre C-Quellcodedatei heißt **matrix.c** und Ihre Assembler-Quellcodedatei heißt **m_add.S**, so können Sie die beiden Dateien folgendermaßen kompilieren/assemblieren und linken:

```
nasm -f elf64 -o m_add.o m_add.S
clang -o matrix m_add.o matrix.c
```

Verwenden Sie die Aufrufkonvention AMD64 ABI.

> Hinweis: Beachten Sie, dass neben Werten auch Pointer beim Funktionsaufruf übergeben werden. Bei diesen Pointern auf die Matrizen handelt es sich um Werte, die Sie direkt als Basis für eine indirekte Adressierung verwenden können. Bedenken Sie dabei auch, dass die Adressierung auf Grundlage von Bytes erfolgt.

Bonus (2 Bonuspunkte)

Über die Mailingliste erhalten Sie Informationen, wie Sie sich insgesamt 2 Bonuspunkte sichern können.

Aufgabe 3 Adressierung in x86-Assembler (4 Punkte)

Häufig müssen Sie in Assembler Daten aus dem Speicher lesen oder in den Speicher schreiben. Während RISC-Architekturen wie MIPS oder ARM oder auch die aus der Systemnahen Informatik bekannte Alpha-Notation dafür nur direkte LOAD- und STORE-Operationen zur Verfügung stellen, bei denen die Adresse fertig berechnet in einem Register liegen muss, bietet x86 sehr komplexe Möglichkeiten, den Speicher zu adressieren. Diese Möglichkeiten wurden Ihnen in Kapitel 1.0 ab Folie 38 vorgestellt.

Im Folgenden sollen Sie stets eine Funktion implementieren, die ein gegebenes Problem lösen soll. Achten Sie darauf, die AMD64-ABI einzuhalten, und beachten Sie die zusätzlichen Hinweise und Einschränkungen zu jeder Aufgabe. Für jedes Problem erhalten Sie außerdem ein Testprogramm auf der Webseite oder eingebettet in der PDF-Datei, mit dem Sie Ihre Funktion linken und testen sollen.

Aufgabe 3.1

`int64_t is_sorted(int64_t* array, uint64_t num_elements)` – Sie erhalten ein Array `array`, das aus vorzeichenbehafteten 64-Bit-Zahlen besteht. Die Anzahl der Elemente des Arrays bekommen Sie über den zweiten Parameter `num_elements` gegeben.

Sofern die Zahlen aufsteigend sortiert sind (nicht streng monoton, also jede Zahl größer oder gleich der vorangegangenen ist), geben Sie `1` zurück; ansonsten geben Sie `0` zurück.

Laden Sie in jedem Schleifendurchlauf das aktuelle Element mit einer einzelnen Instruktion in ein Register. Verwenden Sie die Instruktion `CMP` für den Vergleich. Erklären Sie insbesondere, wie die `CMP`-Instruktion funktioniert.

Aufgabe 3.2

`int64_t dot_product(int64_t* a, int64_t* b, uint64_t num_elements)` – Sie erhalten zwei Vektoren als Arrays von vorzeichenbehafteten 64-Bit-Zahlen `a` und `b`. Die Anzahl der Elemente der Arrays (also die Dimension n der Vektoren) erhalten Sie über den zweiten Parameter `num_elements`. Berechnen Sie das Skalarprodukt der beiden Vektoren und geben Sie es zurück.

Das Skalarprodukt zweier Vektoren a und b ist definiert als die Summe der Produkte der Einträge beider Vektoren, also

$$a \cdot b := \sum_{i=1}^{n} a_i \cdot b_i \text{ für } a, b \in \mathbb{R}^n.$$

Verwenden Sie für die Schleife die `LOOP`-Instruktion. Den Wert des Eintrags aus `a` laden Sie mit der `MOV`-Instruktion; den Wert aus `b` laden Sie direkt im `IMUL`-Befehl. Berechnen Sie dafür die Adresse vorher mit der `LEA`-Instruktion.

Erklären Sie die Funktionsweise der `LOOP`- und `LEA`-Instruktion.

> Hinweis: Bei der Adressierung kann es nötig sein, alle Teile der Adressierung zu verwenden, also
>
> ```
> cmd foo, [base + index * scale + disp] .
> ```

Sie dürfen annehmen, dass die Produkte alle klein genug sind, um in eine vorzeichenbehaftete 64-Bit-Zahl zu passen. Das selbe dürfen Sie für das Ergebnis annehmen, das heißt, Sie müssen keine Overflows und ähnliche Sonderfälle beachten.

Aufgabe 4 Analyse von `fork()` (2 Punkte)

In dieser Aufgabe sollen Sie sich mit der Verwendung der Systemfunktion `fork()` vertraut machen. Betrachten Sie dazu das folgende C-Programm, das Sie auch auf der Website zur Vorlesung herunterladen oder aus der PDF-Datei entnehmen können. Führen Sie es mehrmals hintereinander aus.

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

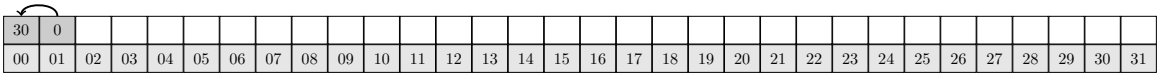
int main() {
        int cnt = 0;
        int fork_pid = getppid();
        printf("cnt = %d, fork_pid = %5d, pid = %5d\n", cnt, fork_pid, getpid());
        fork_pid = fork();
        cnt++;
        printf("cnt = %d, fork_pid = %5d, pid = %5d\n", cnt, fork_pid, getpid());
        fork_pid = fork();
        cnt++;
        printf("cnt = %d, fork_pid = %5d, pid = %5d\n", cnt, fork_pid, getpid());
}
```

1. Zeichnen Sie den Prozessbaum für das C-Programm.
2. Wie viele Ausgabezeilen generiert der Aufruf des Programms? Erläutern Sie anhand des Programmtexts für jede Zeile der bei Ihnen erzeugten Ausgabe, wie sich die Werte für die ausgegebenen Variablen ergeben. Geben Sie dazu die bei Ihnen erzeugte Ausgabe an.
3. Erklären Sie, warum sich die Ausgabe bei wiederholtem Aufruf des Programms manchmal verändert.

Aufgabe 5 Speicherverwaltung (3 Punkte)

In dieser Aufgabe sollen Sie die Verwaltung des Arbeitsspeichers mit `malloc` und `free` übernehmen. Nehmen Sie dafür an, dass der betreffende Teil des Arbeitsspeichers 32 Byte umfasst und byteweise mit Adressen von 0–31 adressiert wird. Programminstruktionen und die unten verwendeten Zeiger (`p`, `q`, ...) werden nicht in diesem Speicherbereich abgelegt, so dass er ganz für `malloc` zur Verfügung steht. Adressen (also auch Zeiger) sind hier 8 Bit (1 Byte) lang. Die Methoden `malloc` und `free` arbeiten wie in der Beispielumsetzung in der Vorlesung vorgestellt. Insbesondere liegt vor jedem von `malloc`/`free` verwalteten Speicherblock ein Header. Dieser Header soll hier aus zwei Byte bestehen: Das erste Byte beschreibt die Größe des jeweiligen Speicherblocks (Header nicht mitgerechnet), das zweite Byte enthält im Falle eines freien Speicherblocks einen Zeiger auf den Header des nächsten Speicherblocks in der Free-Liste.

Zu Beginn besteht der gesamte Speicher aus einem einzigen freien Block, und sieht deshalb folgendermaßen aus:

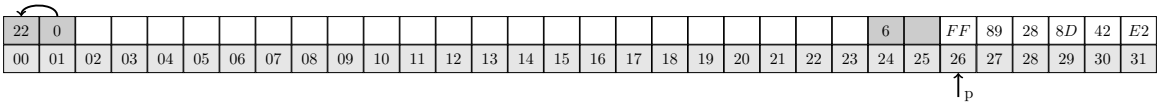


Jedes Kästchen steht für ein Speicher-Byte; bei Kästchen ohne Eintrag ist der Inhalt unbestimmt. Der Übersichtlichkeit halber werden hier die Zeigerfelder freier Speicherblöcke zusätzlich durch Pfeile veranschaulicht.

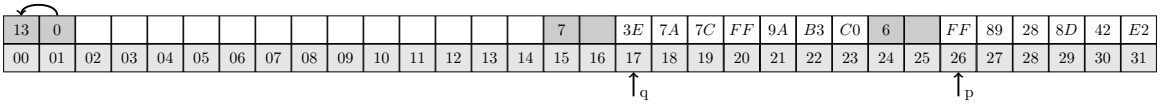
Im Folgenden wird eine Reihe von Anweisungen auf diesem System ausgeführt. Wir geben die Veränderungen, die die ersten drei Schritte im Speicher zur Folge haben, zur Veranschaulichung vor. Setzen Sie dies analog für die darauffolgenden sechs Schritte (d)–(i) fort (0,5 Punkt je Schritt).

> Hinweis: Sie können die Lösung zu dieser Aufgabe in einem (sinnvollen) Dateiformat Ihrer Wahl – Grafikdatei, gescannte handschriftliche Lösung, ASCII-Art, ... – ins Git hochladen. Achten Sie aber bitte darauf, dass die Dateigröße pro Seite unter 5 MB bleibt.

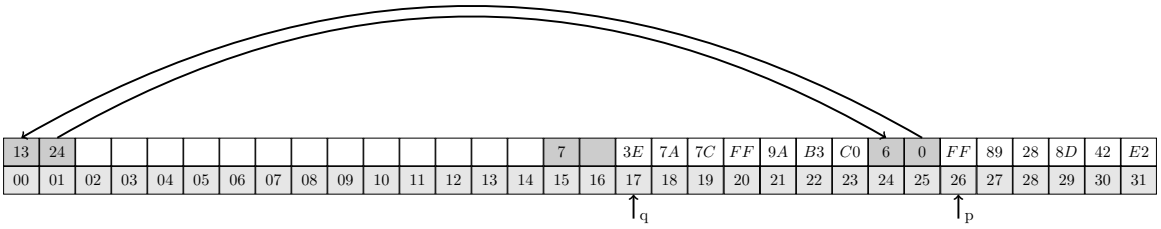
1. `p = malloc(6)` und schreibe `FF 89 28 8D 42 E2` an die Stelle, auf die `p` zeigt



2. `q = malloc(7)` und schreibe `3E 7A 7C FF 9A B3 C0` an die Stelle, auf die `q` zeigt



3. `free(p)`



4. `r = malloc(2)` und schreibe `BE EF` an die Stelle, auf die `r` zeigt
5. `free(q)`
6. `s = malloc(12)` und schreibe `A7 AF C4 7F 03 90 CD 13 A0 F5 49 00` an die Stelle, auf die `s` zeigt
7. `free(p)` – Ja, hier hat der Programmierer wohl einen Fehler gemacht... Machen Sie trotzdem weiter, ein echtes System würde sich davon auch nicht aufhalten lassen!
8. schreibe `8A C3 DF 56 0F 05 FF 10 6E B7 C4 00` an die Stelle, auf die `s` zeigt
9. `t = malloc(15)` und schreibe `4B 61 74 61 73 74 72 6F 70 68 65 21 21 21 00` an die Stelle, auf die `t` zeigt

Hinweis: Sollten Ihnen die Abbildungen in der PDF-Datei zu klein sein, sind die Grafiken als Vektor-Grafiken ebenfalls noch einmal in der PDF-Datei eingebunden.